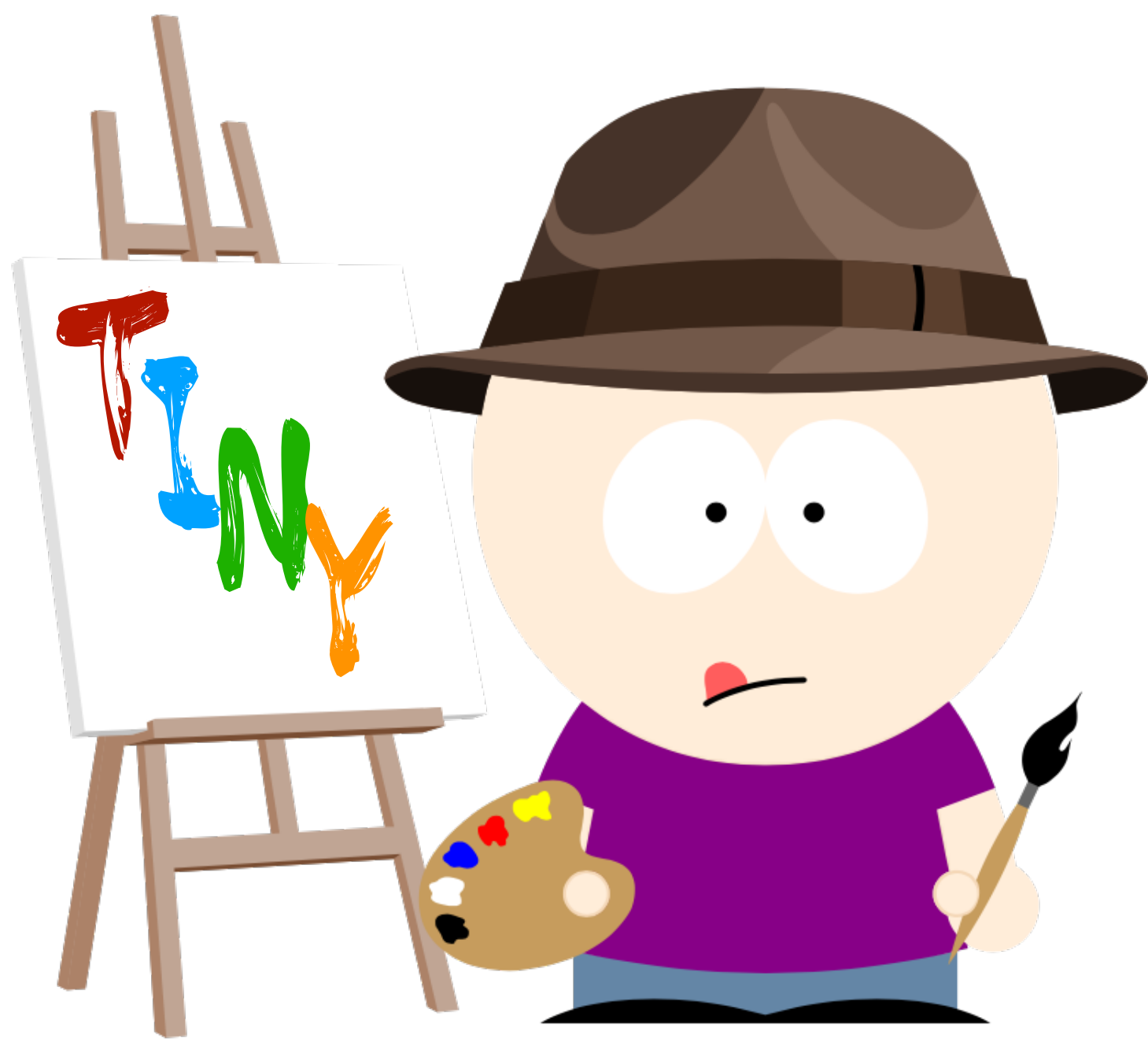




IN-CLASS EXERCISES



PROCEDURE CALLS

Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`prog-1-exercises.zip`

3. Open up a terminal and Unzip the file

`> unzip prog-1-exercises.zip`

`> cd prog-1-exercises`

`> ls`

Task 1

- Show all the recursive steps that the Fibonacci program B.
- Make sure that you include all relevant information in the activation records:
 - return address,^x
 - local variables,
 - actual parameters, etc.
- Make sure you for every recursive step you correctly compute
 - the return address and
 - the return value.

```
int B (int n) {  
    int L,R0,R1;  
(1)  if (n <= 1)  
(2)      L = 1;  
(3)  else {  
(4)      R0 = B(n-1);  
(5)      R1 = B(n-2);  
(6)      L = R0 + R1;  
(7)  };  
(8)  return L;  
}
```

```
int main() {  
(9)  int C = B(4);  
}
```


Task 2

- Draw the stack when control reaches point ● for the third time in this program.
- Include all actual parameters, local variables, return addresses, and dynamic links.
- Also show the stack pointer and frame pointer.

```
void Q (  
    int Y) {  
    int B;  
    B = Y * 2;  
    ●  
    P(B);  
}
```

```
void P(  
    int X) {  
    int A;  
    A = X + 1;  
    Q(A);  
}
```

```
int main () {  
    P(0);  
}
```

Task 3 (a)

- `Godbolt.org` is a site where you can try out different compilers to see what assembly code they generate.
- When you're writing a compiler this can be very useful to help you figure out what kind of code your code generator should generate!
 - Load the code in `task3.c` into `Godbolt.org`.
 - It should look like the screenshot on the next page.

```
void P() {  
}  
  
int main() {  
    P();  
}
```




Code goes here!

C++ source #1

A ▾ 🔒 + ▾ V 🔍

C++ ▾

```
1 void P() {  
2 }  
3  
4 int main() {  
5     P();  
6 }  
7
```

Select this compiler!

mips gcc 15.2.0 (Editor #1) ▾ ×

mips gcc 15.2.0 ▾



Compiler ▾

A ▾ ⚙️ ▾ 📄 🔧 + ▾ 🔪 ▾

```
1 P():  
2     addiu    $sp,$sp,-8  
3     sw       $fp,4($sp)  
4     move     $fp,$sp  
5     nop  
6     move     $sp,$fp  
7     lw       $fp,4($sp)  
8     addiu    $sp,$sp,8  
9     jr       $31  
10    nop
```

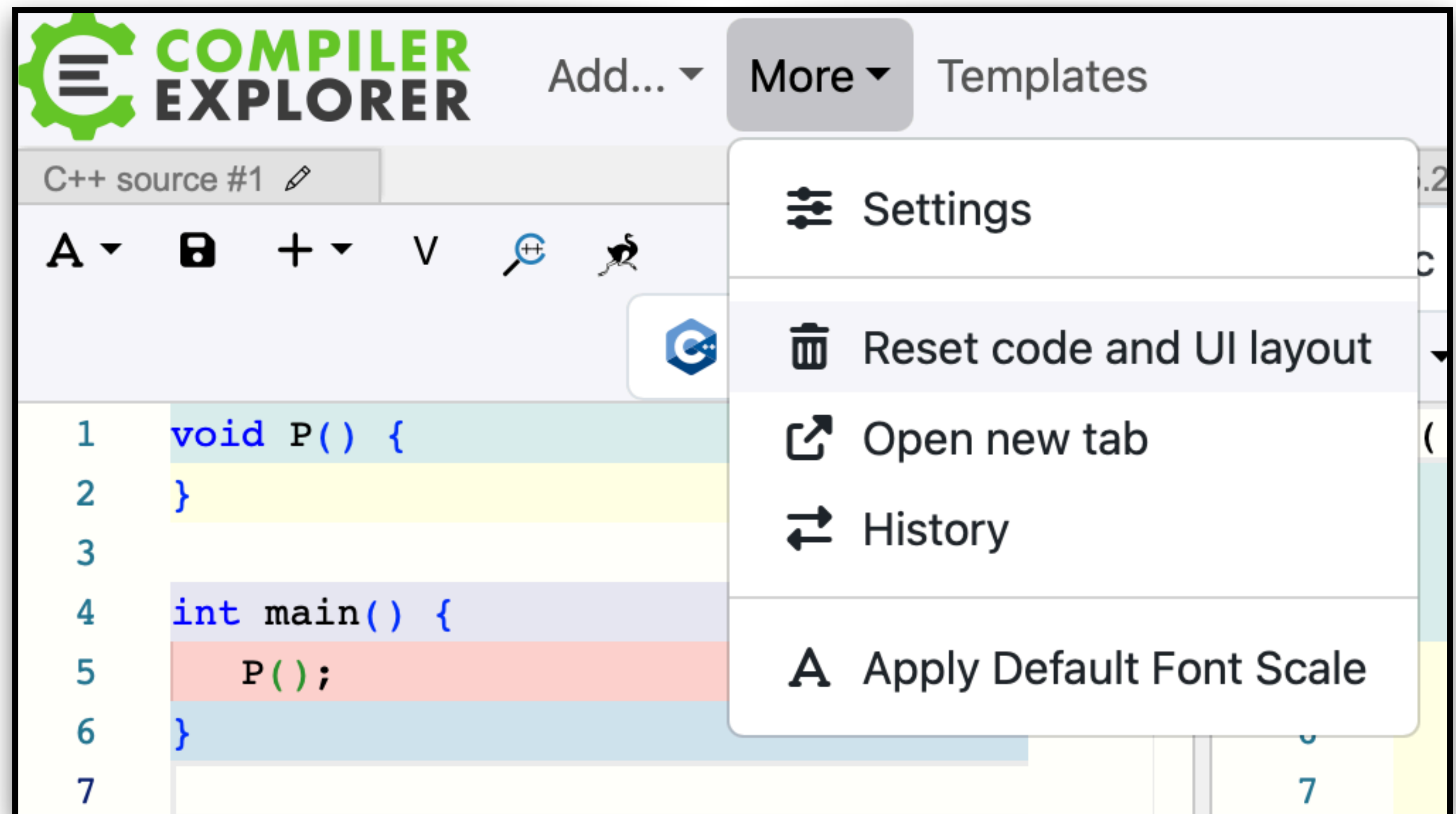
main:

```
13     addiu    $sp,$sp,-32  
14     sw       $31,28($sp)  
15     sw       $fp,24($sp)  
16     move     $fp,$sp  
17     jal      P(.).  
18     nop
```

19

Task 3 (c)

- If you run into trouble, select "Reset code and UI layout" and try again:



Task 3 (d)

- Answer these questions about the generated code:
 1. What is the size of P's activation record?
 2. At what offset is P's frame pointer stored?
 3. Which register stores the return address?
 4. Which instruction returns from P?
 5. Which instruction calls P in main?

```
P:
    addiu    $sp, $sp, -8
    sw       $fp, 4($sp)
    move     $fp, $sp
    nop
    move     $sp, $fp
    lw       $fp, 4($sp)
    addiu    $sp, $sp, 8
    jr       $31
    nop

main:
    addiu    $sp, $sp, -32
    sw       $31, 28($sp)
    sw       $fp, 24($sp)
    move     $fp, $sp
    jal      P
    nop
```


Task 4 (a)

- Load the code in `task4.c` into `Godbolt.org` (also on the next page) and answer these questions:
 1. What is the size of `P`'s activation record?
 2. In which register is 42 passed from `main` to `P`?
 3. At what offset in `P`'s activation record is variable `"a"` stored?

```
void P(int a) {  
    a=67;  
}  
  
int main() {  
    P(42);  
}
```

P:

addiu \$sp, \$sp, -8

sw \$fp, 4(\$sp)

move \$fp, \$sp

sw \$4, 8(\$fp)

li \$2, 67

sw \$2, 8(\$fp)

nop

move \$sp, \$fp

lw \$fp, 4(\$sp)

addiu \$sp, \$sp, 8

jr \$31

nop

main:

addiu \$sp, \$sp, -32

sw \$31, 28(\$sp)

sw \$fp, 24(\$sp)

move \$fp, \$sp

li \$4, 42

jal P

Task 5 (a)

- Load the code in `task5.c` into Godbolt.org (also on the next page) and answer these questions:

1. What is the size of P's activation record?
2. In which registers are arguments 2, 4, 6, 8 passed from main to P?
3. At what offset in P's activation record are formals `a, b, c, d, e` stored?

```
void P(  
    int a, int b, int c,  
    int d, int e) {  
    a=13;  
    b=17;  
    c=19;  
    d=23;  
    e=29;  
}  
  
int main() {  
    P(2,4,6,8,16);  
}
```


P:

```
addiu    $sp,$sp,-8
sw        $fp,4($sp)
move      $fp,$sp
sw        $4,8($fp)
sw        $5,12($fp)
sw        $6,16($fp)
sw        $7,20($fp)
li        $2,13
sw        $2,8($fp)
li        $2,17
sw        $2,12($fp)
li        $2,19
sw        $2,16($fp)
li        $2,23
sw        $2,20($fp)
li        $2,29
sw        $2,24($fp)
nop
move      $sp,$fp
lw        $fp,4($sp)
addiu     $sp,$sp,8
jr        $31
nop
```

main:

```
addiu     $sp,$sp,-40
sw        $31,36($sp)
sw        $fp,32($sp)
move      $fp,$sp
li        $2,16
sw        $2,16($sp)
li        $7,8
li        $6,6
li        $5,4
li        $4,2
jal       P
nop
```




DEFINITIONS AND ALGORITHMS

COLLBERG.CS.ARIZONA.EDU

LIGERLABS.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA